



A Python/Zig optimized and customizable implementation for ρ_{DCCA} and DMC_x^2 coefficients

Fernando Ferraz Ribeiro 
Federal University of Bahia

Gilney Figueira Zebende 
State University
of Feira de Santana

Abstract

This paper presents **Zebende**, a Python package written in Python and Zig, that calculates the *DFA*, *DCCA* functions, and ρ_{DCCA} and DMC_x^2 coefficients, along with auxiliary functions to help the calculations process. The package presents an optimized algorithm that significantly improves the calculations speed in large datasets. **Zebende** is the first package that implements the DMC_x^2 and can be applied to any number of time series. The optimized version of the algorithm is compared to the standard implementation, presenting valuable performance increase. A comparison with other Python and R packages that calculates the ρ_{DCCA} is also presented.

Keywords: ρ_{DCCA} , DMC_x^2 , optimization, Python, Zig.

1. introduction

The Detrended Cross-Correlation Coefficient (ρ_{DCCA}) (Zebende 2011) is a widely used coefficient that measures the cross-correlation between two non-stationary time series. ρ_{DCCA} is an extension of the Detrended Fluctuation Analysis (*DFA*) (Peng, Buldyrev, Havlin, Simons, Stanley, and Goldberger 1994) and the Detrended Cross-correlation Analysis (*DCCA*) (Podobnik and Stanley 2008): while the *DFA* calculates the self-affinity and long-memory properties of a time series data, and the *DCCA* analyses power-law cross correlations between two different non-stationary time series, the ρ_{DCCA} coefficient quantifies this cross-correlation in simple values ranging from -1 to 1 , where -1 indicates a perfect anti-correlation between the series, 1 a perfect correlation and zero (0) no correlation at all.

The method was statistically validated (Podobnik, Jiang, Zhou, and Stanley 2011), applied to climate data Vassoler and Zebende (2012), financial data(Wang, Xie, Chen, Yang, and Yang 2013; Guedes, Dionísio, Ferreira, and Zebende 2017; Ferreira, Dionísio, Guedes, and Zebende

2018), electroencephalogram recorded data (Filho, Ribeiro, Cruz, De Castro, and Zebende 2023), among others. Statistical criteria for evaluating the relevance of the results were also developed (Guedes, Brito, Oliveira Filho, Fernandez, de Castro, da Silva Filho, and Zebende (2018a,b)).

The Detrended Multiple Cross-Correlation Coefficient (Zebende and Silva 2018) (DMC_x^2) is a generalization of the ρ_{DCCA} coefficient that correlates one time series (dependent variable) a number of time series (independent variables). The DMC_x^2 values ranges from zero (0), indicating no multiple cross-correlation to 1, meaning perfect multiple cross-correlation.

The coefficient has been applied to study meteorological data (Brito, Araujo, and Zebende 2019), have been tested statistically (da Silva Filho, Zebende, de Castro, and Guedes 2021), used with the implementation of sliding windows (Guedes, da Silva Filho, and Zebende 2021), to measure the contagion effect on stock market indexes (Guedes, de Castro, da Silva Filho, and Zebende 2022), to produce statistical analysis between stock market behavior (Zebende, Aguiar, Ferreira, and Guedes 2022) or analyze electroencephalogram (EEG) signals (Ribeiro, de Almeida Brito, Filho, Cruz, and Zebende 2025).

This paper presents the **Zebende Python** package, an implementation of the *DFA*, *DCCA*, ρ_{DCCA} , DMC_x^2 and utility functions related to the methods. In section 2 the steps for calculating the ρ_{DCCA} and DMC_x^2 are presented and discussed. Section 3 shows how this library was implemented, the optimization technics and the recommended steps to use the library. In Section 4 the **Zebende** package is compared with other packages for *Python* and *R* that calculates the ρ_{DCCA} in terms of performance and usability, leading to the conclusions in Section 5.

2. Algorithms of the coefficients

The algorithms that calculates the ρ_{DCCA} uses the *DFA* and the *DCCA* steps. The DMC_x^2 coefficient uses the ρ_{DCCA} coefficient and, consequently, also embraces the *DFA* and the *DCCA*. The *DFA* method is described in six steps:

1. Taking a time series $\{x_i\}$ with i ranging from 1 to N , the integrated series X_k is calculated by $X_k = \sum_{i=1}^k [x_i - \langle x \rangle]$ with k also ranging from 1 to N (time series length);
2. the X_k series is divided in $N - n$ boxes of size n (time scale), each box containing $n + 1$ values, starting in i up to $i + n$;
3. for each box, a polynomial (usually of degree 1) best fit is calculated, getting $\tilde{X}_{k,i}$ with $i \leq k \leq (i + n)$;
4. in each box is calculated: $f_{DFA}^2(n, i) = \frac{1}{1+n} \sum_{k=i}^{i+n} (X_{k,i} - \tilde{X}_{k,i})^2$
5. for all the boxes of a time scale, the *DFA* is calculated as:

$$F_{DFA}(n) = \sqrt{\frac{1}{N-n} \sum_{i=1}^{N-n} f_{DFA}^2(n, i)};$$

6. for a number of different timescales (n), with possible values $4 \leq n \leq \frac{N}{4}$ the F_{DFA} function is calculated to find a relation among $F_{DFA} \times n$

The *DCCA* method is very similar to the *DFA* calculations, with the difference of analyzing two series while the *DFA* evaluate properties of a single time series. It's also a six steps process:

1. Taking two time series of the same length $\{x_i^{j1}\}$ and $\{x_i^{j2}\}$ with i ranging from 1 to N , the integrated series $X\alpha_k$ and $X\beta_k$ are calculated by $X_k = \sum_{i=1}^k [x_i - \langle x \rangle]$ for each series, with k also ranging from i to N ;
2. The series X_k^{j1} and X_k^{j2} are divided into $N - n$ boxes of size n (time scale), each box containing $n + 1$ values, starting from i to $i + n$;
3. For each box, a polynomial (usually of degree 1) is fitted, obtaining $\widetilde{X}^{j1}_{k,i}$ and $\widetilde{X}^{j2}_{k,i}$, for series $\{x_i^{j1}\}$ and $\{x_i^{j2}\}$ respectively, with $i \leq k \leq (i + n)$;
4. For each box, one of the $N - n$ boxes of the same temporal scale, the function is calculated by:

$$f_{DCCA}^2(n, i) = \frac{1}{1+n} \sum_{k=i}^{i+n} (X_{k,i}^{j1} - \widetilde{X}_{k,i}^{j1}) \times (X_{k,i}^{j2} - \widetilde{X}_{k,i}^{j2})$$

5. For all boxes of the same temporal scale, the DCCA is calculated as:

$$F_{DCCA}(n) = \sqrt{\frac{1}{N-n} \sum_{i=1}^{N-n} f_{DCCA}^2(n, i)};$$

6. For a number of temporal scales (n), with possible values $4 \leq n \leq \frac{N}{4}$, the DCCA is calculated to find a relationship between $F_{DCCA} \times n$

Comparing the algorithms, the first three are basically identical, the only difference is that the *DCCA* method apply those steps to two series. The step four of the *DFA* can be considered an analogous of the variance, replacing the average subtraction (in the variance) for the values obtained by the polynomial fit (estimated series); and the equivalent step of the *DCCA* is, in the same terms, compared to the covariance between the two series. The technique of fitting a curve, interpreted as a trend inside each box, and subtracting the value estimated by the trend from the actual value in the integrated series ($X_k - \widetilde{X}_k$) from now on will be called **detrended values**(DV). In the *DFA* algorithm, the $f_{DFA}^2(n, i)$ function is the mean of the square of the DV, in *DCCA* calculations, the $f_{DCCA}^2(n, i)$ function evaluates the mean of the product of the DV of the two series in each box.

Step five of the *DFA* calculates the square root of the mean of the values calculates in the previous step for each box, in the *DCCA*, the mean of the values evaluated for each box is calculated in stead. The last step, in both cases, is more a reminder to repeat the respective previous operations for a number of difference time scales (n).

The ρ_{DCCA} is measured by Eq. 1. Considering the relation between *DFA* and variance and *DCCA* and covariance, the ρ_{DCCA} resembles Pearson correlation for a time scale n .

$$\rho_{DCCA}(n) = \frac{F_{DCCA}^2(x\alpha, x\beta)(n)}{F_{DFA}(x\alpha)(n) \times F_{DFA}(x\beta)(n)} \quad (1)$$

The DMC_x^2 is a generalization of the ρ_{DCCA} that calculates the correlation between one time-series $\{Y\}$, as the dependent variable, and a number m of time-series $\{X^1\}$, $\{X^2\}$, $\{X^3\}$,

$\dots, \{X^m\}$ defined as independent variables and represented as $\{X^j\}$, with j ranging from 1 to m . The coefficient is expressed mathematically as:

$$DMC_x^2 \equiv \rho_{Y,X^j}(n)^T \times \rho^{-1}(n) \times \rho_{Y,X^j}(n) \quad (2)$$

In Eq. 2, $\rho^{-1}(n)$ represent the inverse of a matrix populated by all possible combinations of ρ_{DCCA} between independent variables. In Eq. 3, value $\rho_{X^1,X^2}(n)$, for instance, is the ρ_{DCCA} for independent variables X^1 and X^2 calculated with time scale n , occupying position ρ_{12} of the matrix. Two fundamental characteristics: the first is that the main diagonal values are all ones, since it's position in the matrix denotes the calculation of a cross-correlation between a series and itself. Second, the matrix is symmetric in relation to the main diagonal, as the ρ_{DCCA} is a commutative operation.

$$\rho(n) = \begin{pmatrix} 1 & \rho_{X^1,X^2}(n) & \rho_{X^1,X^3}(n) & \dots & \rho_{X^1,X^m}(n) \\ \rho_{X^2,X^1}(n) & 1 & \rho_{X^2,X^3}(n) & \dots & \rho_{X^2,X^m}(n) \\ \vdots & \vdots & \vdots & \dots & \vdots \\ \rho_{X^m,X^1}(n) & \rho_{X^m,X^2}(n) & \rho_{X^m,X^3}(n) & \dots & 1 \end{pmatrix} \quad (3)$$

At last Eq. 4 represent the transposed vector of the $\rho_{Y,X_i}(n)$ between the depended variable $\{Y\}$ and each $\{X_i\}$ independent variable for a given time scale n .

$$\rho_{Y,X^j}(n)^T = [\rho_{Y,X^1}(n), \rho_{Y,X^2}(n), \dots, \rho_{Y,X^m}(n)] \quad (4)$$

As the *DFA* and the *DCCA*, ρ_{DCCA} and DMC_x^2 should be evaluated in a number of time scales (n) to analyze the characteristics of each coefficient.

3. Zebende package: implementation and optimization

The implementation of the **Zebende** package follows some well defined goals:

1. Enhance performance;
2. avoid redundant calculations;
3. make the outputs compatibles with other data analyses tools (including data manipulation, machine learn and statistical packages);
4. manage multiple time series inputs;
5. operate the DMC_x^2 for any number of series;
6. create a customizable and modular set of tools;
7. facilitate package evolution and maintenance;
8. deliver an easy to use package.

The Python language was chosen because it is one of the most used languages in the data analyses field and have a great support for statistical tools and machine learning algorithms. There are a plethora of tools to load and manipulate data (**Pandas**, **Polar**, **PySpark** ...), execute statistical analyzes (**Numpy**, **SciPy**, **StatsModels** ...), machine learning (**Pytorch**, **TensorFlow**, **Scykit Learn** ...) and data visualization (**Matplotlib**, **Seaborn**, **Plotly** ...) among other data related applications.

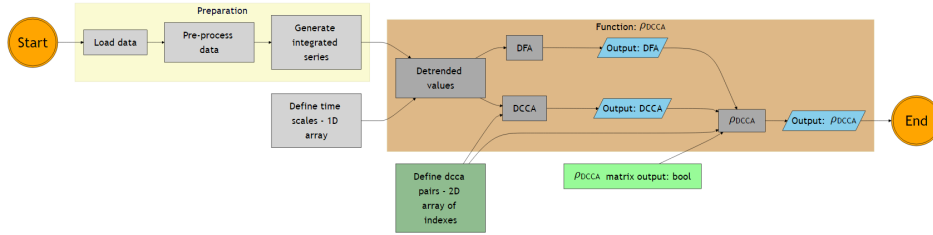


Figure 1: Calculating ρ_{DCCA} with **Zebende** package - Simplified flowchart

The first draft of the code was written in pure Python, to rapidly prototype the way users will interact with the package. Figures 1 and 2 presents simplified flowcharts illustrating how to use the package and how the main functions (ρ_{DCCA} and DMC_x^2) works.

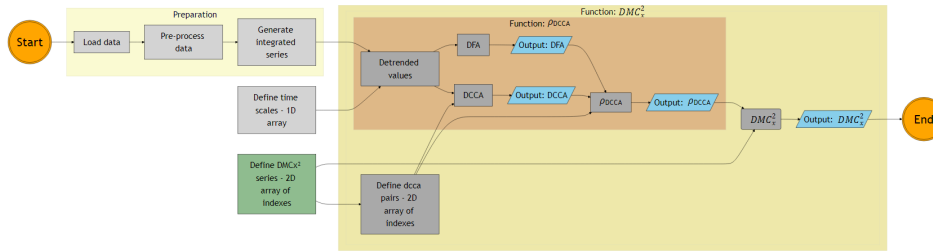


Figure 2: Calculating DMC_x^2 with **Zebende** package - Simplified flowchart

The preparation steps are the same in both functions. First the data is loaded, and should be analyzed by the researchers. Based on the data characteristics, the set should be treated to ensure the methods requirements in the "Pre-processing" stage. The package functions expects data as a matrix with the columns as the series and the lines as time steps. Columns unwanted in the indented research should also be dropped for better performance of the algorithms in this step. The more common way to do that is to use a data manipulation package. To proceed to the next step, the data table should be in the form of a **Numpy** 2D array. Any data manipulation Python package can export a table as a **Numpy** array. The next step is to calculate the integrated series. The package provides a function, named `integrated_series()`, to calculate that. The code example below show how to load the libraries (using **Pandas** as the data manipulation packages and loading a `.csv` file as a generic example), convert to **Numpy** array and generate the integrated series.

```
# importing packages
import numpy as np
import pandas as pd
import zebende as zb
```

```

data = pd.read_csv('path_to_the_file.csv') # loading data
# Pre-processing data
# ...
data = data.to_numpy(data) #converting data to Numpy array
int_data = zb.integrated_series(data) # calculating the integrated series

```

The option of taking out the integrated series generation from the main methods (`p_dcca()` and `dmcx2()`) to an independent one was inspired by [Peng *et al.* \(1994\)](#) work, where the way of calculating integrated series was different from the one that is widely used in more recent years. The integration of the series is essentially a pre-processing step, and this approach makes easy to explore alternative ways to integrate the series or compare [Peng *et al.* \(1994\)](#) approach to the current most used one in different scenarios, or even embrace new proposals for the series integrating step.

The input and output structures of each function are displayed below:

```

def p_dcca(
    input_data: NDArray[float64],
    tws: NDArray[int64] | NDArray[float64],
    DCCA_of: ndarray | Literal['all'] = "all",
    P_DCCA_output_matrix: bool = False
) -> tuple[NDArray[float64], # DFA
           NDArray[float64], # DCCA
           NDArray[float64]  # P_DCCA
          ]

def dmcx2(
    input_data: NDArray[float64],
    tws: NDArray[int64] | NDArray[float64],
    dmcx2_of: ENUM_DMCx2_of | NDArray[float64] | list = 'all-full'
) -> tuple[ NDArray[float64], # DFA
           NDArray[float64], # DCCA
           NDArray[float64], # P_DCCA
           NDArray[float64], # DMC
          ]

```

The first two inputs are the same for functions `p_dcca()` and `dmcx2()`: `input_data` receive the integrated series and the `tws` receives an 1D array representing the time scales (box size) described in the algorithms on Section 2. The `input_data` is a 2D array of 64 bits floating point data. The `tws` accepts integers and, for convenience reasons, also floating points. Since the size of the boxes needs to be integers, in case of floating points, the values will be converted to integers by ignoring the decimal values (truncating). This two inputs are colored in light gray in Figures 1 and 2, indicating mandatory inputs.

With the mandatory steps explained, some very important optional inputs should be addressed. Starting with the ρ_{DCCA} function (dark green node in Figure 2) represents the input `DCCA_of` of the function. This input requires a 2D array of integers, each row is a pair of

index, related to the `data_input` matrix. For example: if the `input_data` receives a four columns matrix, with index ranging from 0 to 3, that is intended to calculate de *DFA* for all the series but the ρ_{DCCA} between series of index 0 and 1 and also for series 2 and 3, the `DCCA_of` input should receive the array `[[0,1], [2,3]]`. If no value (or the string 'all') is given, the function will calculate all possible combinations of *DCCA* calculations between all the series respecting the index values order, as below:

```
[[0,1], [0,2], [0,3], [1,2], [1,3], [2,3]]
```

3.1. Implementation of the Detrended Cross-correlation Coefficient function

Is very important to understand the calculation steps, the role of the `DCCA_of` array and how it fits in the goals of the package implementation. The code below is part of the pure Python implementation of the ρ_{DCCA} function,

```
for n_index, n in enumerate(tws): # for each time scale
    # temporaty allocation arrays
    f2dfa_n = np.full(shape=(data.shape[0] - n, data.shape[1]),
        fill_value=np.nan, dtype=data.dtype)
    dcca_n = np.full(shape=(data.shape[0] - n, DCCA_of.shape[0]),
        fill_value=np.nan, dtype=data.dtype)
    detrended_mat = np.full(shape=(n + 1, data.shape[1]),
        fill_value=np.nan, dtype=data.dtype)
    for i in range(data.shape[0] - n): # for each box
        detrended_series( # inputs
            time_steps[i : i + (n + 1)], # arr_x
            data[i : i + (n + 1), :], # mat_y
            detrended_mat, # output
        )
        f2dfa_n[i] = np.power(detrended_mat, 2).mean(axis=0)
        for j, pair in enumerate(DCCA_of): # for each DCCA pair
            dcca_n[i, j] = (detrended_mat[:, pair[0]] * detrended_mat[:, pair[1]]
                ).mean(axis=0)
    F_DFA_arr[n_index, :] = np.sqrt(f2dfa_n.mean(axis=0))
    DCCA_arr[n_index, :] = dcca_n.mean(axis=0)
    # calculation of P_DCCA
    P_DCCA_output_function(n_index, DCCA_of, F_DFA_arr, DCCA_arr, # Inputs
        P_DCCA_arr) # Output
```

The first `for` loop in the code operates over the values of the `tws` input, ensuring that step 6 of the *DFA* and *DCCA* methods, presented in Section 2, is being carried out. In other words, the calculations will be applied, sequentially, to every single value in the `tws` array. Three temporary arrays are allocated and resized for each time scale. The first, `f2dfa_n`, is used to store the calculations of the step 4 of the *DFA* (f_{DFA}^2). The number of lines of this array correspond to the number of boxes in the current time scale ($N - n$, resized for each

time scale n) and the columns is the number of series in the analysis (same size for every value of n). Second, the array `dcca_n` holds the values calculated in the step 4 of the *DCCA* (to calculate $f_{DCCA}^2(n, i)$). The number of lines also correspond to the number of boxes(resized for each n) but the number of columns equals the number of pairs (rows) in the *DCCA_of* input (same size in each n). The `detrended_mat` array has a number of lines equal to the number of points in a time scale box ($n + 1$, resized for each n) and column count also equal to the number of time series (same size in each n). The first two temporary arrays will store data for all the boxes in the time scale, the last one will be used in each box and will have the values replaced in the next one, until the last box of the time scale. Than all the arrays will be cleared and recreated with shapes calculated with the new value of n .

After the allocation of the temporary arrays, the second `for` loop operates in every box for a certain time scale n . In each box, the `detrended_series()` function execute a one degree polynomial fit, subtract the values of the series with the value given by the interpolated curve(DV) and stores this values in the `detrended_mat`. After that, the **mean of the square of each DV in the box** ($f_{DFA}^2(n, i)$) is calculated in the current box for every time series and stored in the `f2dfa_n` array in the line associated with the current box, and the column related to each time series.

The next nested `for` loop operates over the *DCCA_of* array. For each line (pair) in the *DCCA_of* array, the $f_{DCCA}^2(n, i)$ is evaluated by multiplying the corresponding DV from the two series boxes in the current pair, getting the mean, and stores it in the `dcca_n` temporary array.

After all the pairs are calculated, the algorithm goes back to the each box `for` loop and head to the next box for the current time scale. when all the all the boxes are calculated, the final results for the *DFA*, *DCCA* and ρ_{DCCA} are evaluated and saved in the output vectors `F_DFA_arr`, `DCCA_arr` and `P_DCCA_arr` respectively.

The function named `P_DCCA_output_function()` in the code above, is a pointer to other functions. One that outputs the ρ_{DCCA} results in the form of a table (rows for each time scale and columns for each *DCCA_of* pair) and the other outputs it the form of a 3D matrix, where each level is the matrix in Equation 3 for one of the time scales. This behavior is driven by the `P_DCCA_output_matrix` input (represented as the light green node in Figure 1), where `False` means table output and `True` matrix output. This is very convenient for calculating the DMC_x^2 . There are two utility functions to transform a table output in a matrix one (`p_dcca_simple_to_matrix()`) and also the other way around (`p_dcca_matrix_to_simple()`).

3.2. Implementation of the Detrended Multiple Cross-correlation Coefficient function

The `dmcx2()` function runs the `p_dcca()` with `P_DCCA_output_matrix` set to `True` in the background. There is no *DCCA_of* input for the DMC_x^2 function, instead there is a `dmcx2_of` parameter. This input receives a 2D matrix where each line represents the indexes of the series to be used in Equation 2. For each row, the first elements is the index of the series used as the dependent variable, the others, the index of the independent ones. There are two literal strings that can be used, for convenience as inputs for this parameter: `'all-full'`, that generates a 2D array with every series as the dependent variable against all the others; and `'first-full'`, with only one row, having the index zero series as the dependent variable in relation with all the others. The `'all-full'` option is conducted by calling `dmc_of_all_as_y()`, also

available as a utility function.

With a given `dmcx2_of` an array with all the necessary pairs for the *DCCA* and ρ_{DCCA} is automatically generated and used for the background `p_dcca()` function to calculate da matrix as in Eq. 3. The `dcca_of_from_dmcx2_of()`, also can be used as an utility function, receives the `dmcx2_of` as input and returns the `dcca_of` array. With the matrix ρ_{DCCA} assembled, two internal functions, that can also be used as utility functions, calculates the DMc_x^2 for all the lines in the `dmcx2_of` matrix, for all the time scales. The first function, `dmcx2_from_p_dcca_matrix()`, that receives the ρ_{DCCA} and the `dmcx2_of` array, is presented in the code below.

```
def dmcx2_from_p_dcca_matrix(P_DCCA_arr: NDArray[np.float64],
    dmcx2_of: NDArray[np.float64]) -> NDArray[np.float64]:
    # DMCx2 output matrix
    DMCx2_arr = np.full(shape=(P_DCCA_arr.shape[2], dmcx2_of.shape[0]),
        fill_value=np.nan, dtype=P_DCCA_arr.dtype)

    for n_index in range(P_DCCA_arr.shape[2]):
        P_DCCA_arr_2D = P_DCCA_arr[:, :, n_index]
        for j, dmcx2_of_1D in enumerate(dmcx2_of):
            DMCx2_arr[n_index, j] = dmcx2_from_p_dcca_matrix_2d(P_DCCA_arr_2D,
                dmcx2_of_1D)

    return DMCx2_arr
```

The `dmcx2_from_p_dcca_matrix()` function executes a for loop over the ρ_{DCCA} that separates the this 3D matrix in to the 2D matrices for each time scale. Nested in this loop, another for extracts each line of the `dmcx2_of` and passes, the extracted matrix and the line vector to the `dmcx2_from_p_dcca_matrix_2d()`, displayed here.

```
def dmcx2_from_p_dcca_matrix_2d(P_DCCA_arr_2D: NDArray[np.float64],
    dmcx2_of_1D: NDArray[np.float64]) -> NDArray[np.float64]:
    y_index = dmcx2_of_1D[0:1]
    x_indexes = dmcx2_of_1D[1:]

    mat_x = P_DCCA_arr_2D[np.ix_(x_indexes, x_indexes)]
    vec_y = P_DCCA_arr_2D[np.ix_(x_indexes, y_index)]

    return vec_y.T @ np.linalg.inv(mat_x) @ vec_y
```

The `dmcx2_from_p_dcca_matrix_2d()` function uses **Numpy** methods to prepare the data to apply Eq. 2. The list of indexes is divided in `y_index`, holding an one item array whit the index of the dependent variable time series, and `x_indexes`, containing the indexes of the independent ones. The **Numpy** `np.ix_()`, although it's not a very known function of the library constructs index arrays that will use the cross product from a series of 1D arrays as inputs. It's a very convenient way to extract a sub matrix and a vector from the ρ_{DCCA} matrix. The matrix, as assembled by the `p_dcca()` function, will always need to have extractions of a 2D matrix and a vector, as we can see in Eq. 2. The code below is an example of the extraction process.

```
import numpy as np
arr = np.array([[1,2,3,4],
                [2,1,5,6],
                [3,5,1,7],
                [4,6,7,1]])

print(arr)
```

```
[[1 2 3 4]
 [2 1 5 6]
 [3 5 1 7]
 [4 6 7 1]]
```

An illustrative `arr` matrix is defined as 4×4 with all ones in the main diagonal and symmetric integer values in the other cells. Since the ρ_{DCCA} ranges from -1 to 1 , those values should be interpreted as place holders.

```
sub_mat_index = np.ix_([1,2,3], [1,2,3])
print("index combination:\n", sub_mat_index)
print("extracted matrix:\n",arr[sub_mat_index])
```

```
index combination:
(array([[1],
        [2],
        [3]]), array([[1, 2, 3]]))
extracted matrix:
[[1 5 6]
 [5 1 7]
 [6 7 1]]
```

Above, a sub matrix, holding the positions 1 to 3 is extracted using `np.ix_()` function, and below, the extraction of the vector, first as a line and then as a column vector.

```
sub_mat_vec = np.ix_([0], [1,2,3])
print("line vector:\n", arr[sub_mat_vec])
sub_mat_vec = np.ix_([1,2,3], [0])
print("column vector:\n", arr[sub_mat_vec])
```

```
line vector:
[[2 3 4]]
column vector:
[[2]
 [3]
 [4]]
```

In the **Zebende** package, the vector is extracted as a column for better coherence with the DMC_x^2 theory. The method also works for dependent variable different of index 0. The resulting matrix will preserve the diagonal as 1, the symmetry regarding the main diagonal and the order of elements in the column vector respected. the code below extract the index 1 series as the dependent and the others as independent.

```

sub_mat_index = np.ix_([0,2,3], [0,2,3])
print("index combination:\n", sub_mat_index)
print("extracted matrix:\n",arr[sub_mat_index])
sub_mat_vec = np.ix_([0,2,3],[1])
print("column vector:\n", arr[sub_mat_vec])

index combination:
(array([[0],
       [2],
       [3]]), array([[0, 2, 3]]))
extracted matrix:
[[1 3 4]
 [3 1 7]
 [4 7 1]]
column vector:
[[2]
 [5]
 [6]]

```

The idea of separating the calculations of the ρ_{DCCA} calculations in three distinct functions aims to different workflows. The `dmcx2()` function calculates and outputs all the prerequisites, as the DFA , $DCCA$, ρ_{DCCA} along with the DMC_x^2 . This is the most practical way of getting all this calculations conducted. But the task can also be divided in two: first use the `p_dcca()` function to generate DFA , $DCCA$, ρ_{DCCA} outputs, analyze the outputs and then get the DMC_x^2 using `dmcx2_from_p_dcca_matrix()`.

Function `dmcx2_from_p_dcca_matrix_2d()` can be used to more customizable applications. Imagine a use case where only the ρ_{DCCA} anti-correlation, inside a certain range, in relation to the dependent variable, with the `dmcx2_of` set to `all-full`. From previous ρ_{DCCA} studies is known that this coefficient can vary from positive to negative and vice versa in different time scales. This implies that the ρ_{DCCA} matrix should be analyzed in every time scale from every line of the `dmcx2_of`, also implies that the `dmcx2_of` may not be a matrix in the since that the rows may have different. The **Numpy** ND Array could not hold that. Many different workarounds could be proposed for that situation. In the implementation of this package, function `dmcx2_from_p_dcca_matrix_2d()` allow the user to make a custom code that extracts the ρ_{DCCA} matrix for the current time scale and extracts the elements that fit the rules from each `dmcx2_of` row.

3.3. Zig implementation

The Python implementation successfully reflects the implementation goals presented in Sec. 3 but the performance could benefit with the integration of a low-level language. **Zig** was chosen to enhance the algorithms performance. It is a low-level language that gain popularity in recent years and provides performance similar to **C** and **Fortran** in some scenarios (Kacs, Lee, Zarins, and Brown 2024).

The interest in using **Zig** language for this project also relies on the cross-compiling capabilities of the **Zig** compiler. For a small research group maintaining a package could be challenging and the ability to to compile all releases for all platforms in a single machine is a great

advantage. The Zig compiler can generate binaries for Windows, Linux and MacOS from a single machine. The Zig compiler is also very fast, and the language is very easy to learn, with a syntax that is very similar to C.

The implementation focus on writing technics ρ_{DCCA} function in Zig exposed as a C Application Binary Interface (ABI), called by Python using the **ctypes** package. The ρ_{DCCA} function is the most computational expensive part of the package, and the performance gain was expected to be significant.

The output arrays are allocated in the Python side and passed as pointers to the Zig function together with the series matrix, **tw**s and **DCCA_of** arrays. The Zig function receives the pointers and the size of the arrays as inputs, and the results are stored in the same memory space. Before passing to Zig must be assured that the arrays are contiguous, using the **np.ascontiguousarray()** function. The boolean parameter **P_dcca_matrix_output** is also passed to the Zig function, to determine if the output of the ρ_{DCCA} should be a table or a matrix.

The Zig implementation follows the same steps as the Python one, but respecting languages differences. In Python, using **Numpy**, all the time series calculations occur in the same line of code, using the **Numpy** broadcasting capabilities. In Zig, the calculations are made in a nested for loop.

3.4. Algorithm optimization

Although the Zig implementation presents a significant performance gain, the algorithm still can be optimized. The strategy is to focus on avoiding repeated calculations in the process. The code below shows the calculations of the polynomial fit before the optimization.

```
pub fn lin_ls_fit(win: []f64, time: []f64) [2]f64 {
    var x_sum: f64 = 0;
    var y_sum: f64 = 0;
    var xy_sum: f64 = 0;
    var x2_sum: f64 = 0;
    for (win, time) |w, t| {
        x_sum += t;
        y_sum += w;
        xy_sum += t * w;
        x2_sum += pow(f64, t, 2);
    }
    const n: f64 = @as(f64, @floatFromInt(time.len));
    //slope
    const slope: f64 = (((n * xy_sum) - (x_sum * y_sum)) /
        ((n * x2_sum) - (pow(f64, x_sum, 2))));
    //inter
    const inter: f64 = ((y_sum - (slope * x_sum)) /
        (n));
    //result
    return [_]f64{ slope, inter };
}
```

The code above calculates the slope and the intercept of a linear least squares fit. The function runs on every box for every time scale. In the optimized version, for consecutive boxes, the value of the sums from the previous box is used to calculate the next one without a loop for every item in the box. This technic proved to be very efficient in increasing performance. the expressions below show the effectiveness of the optimized algorithm.

$$\forall 1 < i \leq (N - n), \sum_{k=i}^{i+n} T_k = \left(\sum_{j=i-1}^{(i+n)-1} T_j \right) - T_{i-1} + T_{i+n} \quad (5)$$

$$\forall 1 < i \leq (N - n), \sum_{k=i}^{i+n} T_k^2 = \left(\sum_{j=i-1}^{(i+n)-1} T_j^2 \right) - T_{i-1}^2 + T_{i+n}^2 \quad (6)$$

$$\forall 1 < i \leq (N - n), \sum_{k=i}^{i+n} S_k = \left(\sum_{j=i-1}^{(i+n)-1} S_j \right) - S_{i-1} + S_{i+n} \quad (7)$$

$$\forall 1 < i \leq (N - n), \sum_{k=i}^{i+n} (S_k \times T_k) = \left(\sum_{j=i-1}^{(i+n)-1} (S_j \times T_j) \right) - (S_{i-1} \times T_{i-1}) + (S_{i+n} \times T_{i+n}) \quad (8)$$

The expressions (Eq. 5, Eq. 6, Eq. 7 and Eq. 8) showcases how the optimizations can be implemented in the code. Each expression display that the sum of the values in a box can be calculated using the sum of the previous box and the values of the first and last elements of the current box. Eq. 5 presents the sum of the time stamps values and Eq. 6 the sum of the squares of the time stamps values. This calculations occur once for every box in every time scale and is stored in variables. Eq. 7 and Eq. 8 presents the same calculations for the series values. Storing values for each time series.

Also, to optimize the calculations from one time scale to the next, the first sums for the first box are saved in a temporary variable. Considering the current time scale as `tw_prev` and the consecutive as `tw_current`. To calculate the current value, the algorithm takes the sums from the `tw_prev` and add the values with indexes that exceed the `tw_prev` size. The code was also rewritten with structs, to hold the temporary values and the functions for better readability. The code below shows the optimized version of polynomial fitting calculations.

```
fn shiftWindow( self: *MainOperator,
               n: usize, win_start: usize,
               F_DFA_ptr: *allowzero [*c]f64) void {
    self.time_window = self.time[win_start ..] [ .. (n + 1)];
    // print("win_start {}\n", .{win_start});
    if (win_start != 0) {
        // updating sum_x
        self.current.sum_x = self.current.sum_x - self.left_x + self.time_window[n];
        // updating sum x^2
        self.current.sum_x2 = self.current.sum_x2 -
            pow(f64, self.left_x, 2) + pow(f64, self.time_window[n], 2);
```

```

// updating y and y*x for every serie
for (self.series, 0..) |serie, sr_index| {
    serie.current.sum_y = serie.current.sum_y -
    serie.left_y + serie.series[win_start + n];

    serie.current.sum_xy = serie.current.sum_xy -
    (self.left_x * serie.left_y) +
    (self.time_window[n] * serie.series[win_start + n]);
    serie.left_y = serie.series[win_start];
    self.detrended(serie, win_start, &F_DFA_ptr.*[sr_index]);
}
} else { // win_start == 0
    self.current.window_len = n + 1;

    for (self.previous.window_len..self.current.window_len) |i| {
        self.previous.sum_x += self.time_window[i];
        self.previous.sum_x2 += pow(f64, self.time_window[i], 2);

        for (self.series) |serie| {
            serie.previous.sum_y += serie.series[i];
            serie.previous.sum_xy += self.time_window[i] * serie.series[i];
        }
    }

    // updating current sum values
    self.current.sum_x = self.previous.sum_x;
    self.current.sum_x2 = self.previous.sum_x2;

    for (self.series, 0..) |serie, sr_index| {
        serie.current.sum_xy = serie.previous.sum_xy;
        serie.current.sum_y = serie.previous.sum_y;

        serie.left_y = serie.series[win_start];
        self.detrended(serie, win_start, &F_DFA_ptr.*[sr_index]);
    }
}
self.left_x = self.time_window[0];
}

```

In the code above, the `MainOperator` is a structure that holds the time series (Y_n), the time stamps (x), the sums for the time window and for each series and the temporary variables that holds the first values of the previous box. The `shiftWindow()` function is a method of the `MainOperator` structure. The function receives the size of the box, the index of the first element of the box, and a pointer to the F_{DFA}^2 array. The function is called for every box in every time scale. The function is responsible for updating the sums of the time series, the sums of the DV and the DV matrix.

The if statement in the code above is responsible for updating the sums for the current box.

The first `if` block is executed for every box except the first one in each time scale, where the `else` block is executed instead.

The `if` block first update the values for the sum of the time stamps (`sum_x`) and the sum of the squares of the time stamps (`sum_x2`). Then, for every series, the sum of the series values (`sum_y`) and the sum of the product of the series and the time stamps are updated (`sum_xy`). Then the values are passed to the `detrended` function that calculates The DV and the DFA for the current box.

The `else` block is executed for the first box in each time scale. For consecutive time scales, the values of the sums are updated using the values of the previous box, looping and adding only the values in the current box that don't overlap with the previous one. The `detrended` function is also called for every series in the time scale.

The three implementations, pure Python, Zig and optimized Zig, were tested and the results are presented in the next section.

4. Results

The algorithm was tested with data and results from the Ribeiro *et al.* (2025) paper. Details regarding data preparations can be found in the aforementioned paper. A comprehensive dataset of EEG recording, with 108 subjects, executing 12 experiments each, with the ρ_{DCCA} and DMC_x^2 calculated for all experiments, using signals from four electrodes with 15742 points. The results were compared with the original ones, and the algorithms were validated. The original data came from the Schalk, McFarland, Hinterberger, Birbaumer, and Wolpaw (2004) paper, available on the *Physionet* open-access website, that can be accessed in this link:

<https://physionet.org/content/eegmidb/1.0.0/>

Regarding performance, the pure Python implementation, the Zig implementation and the optimized Zig were compared with 3 other packages that calculate the ρ_{DCCA} . Researching the R CRAN repository, the **DFA** (Mesquita, Filho, and Danilewicz. 2024), a pure R implementation, and the **DCCA** (Prass and Pumi 2020), a R package implemented in Java were found. From the Python **pypi** repository, a package also named **DCCA** (Alchieri, Abdalazim, Alecci, Gashi, Gjoreski, and Santini 2024), a Python implementation with massive use of **Numpy** broadcasting capabilities and memory access with strides to avoid loops, was also used. All those packages have ρ_{DCCA} functions. Hereafter the **DCCA** R-java package will be referred as **DCCA-R-Java** and the **DCCA** Python package as **DCCA-Python** for better distinction.

The dmc_x^2 is a more recent coefficient, and no other package was found that calculates it besides the **Zebende** package. That alone is a huge contribution of this package. The ρ_{DCCA} is a fundamental step to calculate the DMC_x^2 and contains the most computationally expensive parts. In the DMC_x^2 calculation, the ρ_{DCCA} is calculated for, at least, a combination of 3 series two by two. With the idea to broadly compare the performance of the **Zebende** package with the others, aiming an overall order of magnitude of the packages compared speed, using the same computer and the same time window scales, the performance comparison was made using data from one of the experiments described in the Schalk *et al.* (2004) paper, with 19920 lines and 64 channels. For convenience the data was posted in the link below:

<https://tinyurl.com/zbtestdata01>

None of the comparison packages can calculate multiple series comparison in the same input, so the ρ_{DCCA} methods were wrapped around a loop that alternates the series used by the function, using a four two by two combination of the data indexes. The **DCCA-Python** package also didn't accept multiple inputs for the time scale, so another loop was added in that test.

First all the packages were used to calculate the ρ_{DCCA} of combinations of four time series. The results are presented in the Table 1.

Table 1: Elapsed Time Comparison with four series

Package	Implementation	Elapsed Time(s)	Performance Increase
Zebende	Zig (opt)	35.88	-
Zebende	Zig (basic)	59.74	66.50%
DCCA	Python	447.67	1,147.69%
DCCA	R-Java	2,206.2	6,048.83%
Zebende	Python	3,076.23	8,473.66%
DFA	R	28,116.00	78,261.20%

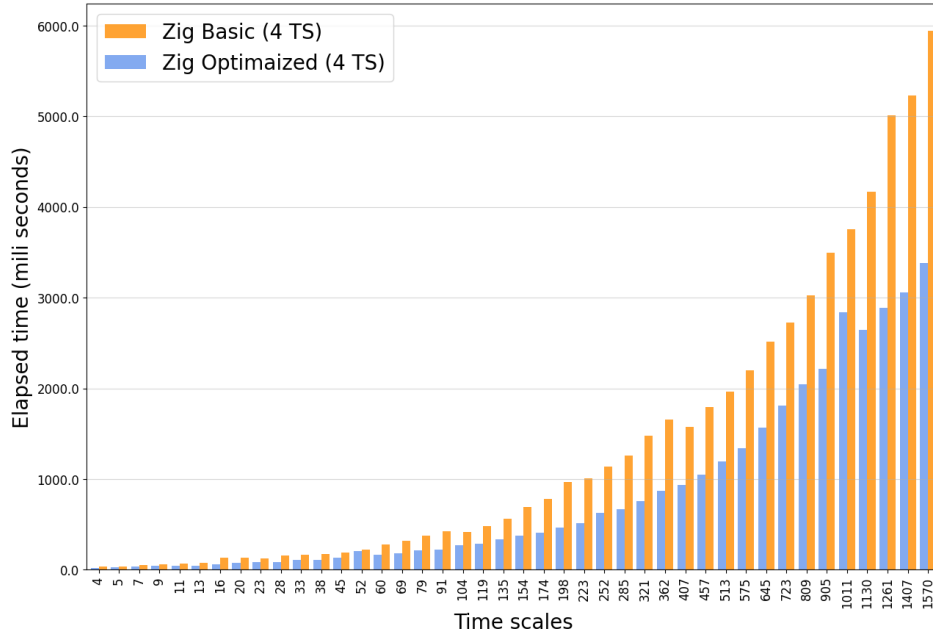


Figure 3: Elapsed Time Comparison for Different Implementations (four time series)

The **Zebende** package, using the optimized Zig implementation, was the fastest, with a 27.52 seconds elapsed time. The basic Zig implementation was the second fastest, with 42.03 seconds. The **DCCA-Python** package was the third fastest, with 370.54 seconds. The **DCCA-R-Java** package was the fourth fastest, with 1987.67 seconds. The **Zebende** package, using the Python implementation, was the fifth fastest, with 2415.94 seconds. The **DFA-R** package was the slowest, with 20173.13 seconds.

The **Zig** basic and optimized versions occupying the first two positions in the ranking is not unexpected, hence the **Zig** language is more performative than the other ones used in the comparison. The **DCCA-Python** package in the third position is a surprise, showing that a well implemented **Python** package can overperform a **Java** implementation.

The 66.50% increase in performance in the basic compared to the optimized **Zig** implementation is a good result, showcasing the effectiveness of the optimizations. Another test was conducted, with the same data and settings as the previous one, but getting the elapsed time inside every time scale. These results are presented in Fig. 3.

Figure 3 shows the elapsed computation time for the Basic and the optimized version of the algorithm. The differences between the two implementations increases as the time scale increases. If the test was conducted with a bigger time series, and with larger time scales the difference would be even more significant.

The **Zebende** package implementations were designed to increase performance in scenarios with large number of points in the series, but also with a large amount of time series used in the DMC_x^2 calculations. To prove that, the same tests were also made using the same data, with the combinations of twelve series, 2 by 2. The results of that second test are presented in Table 2.

Table 2: Elapsed Time Comparison with twelve series

Package	Implementation	Elapsed Time(s)	Performance Increase
Zebende	Zig (opt)	120.55	-
Zebende	Zig (basic)	213.08	76.64%
Zebende	Python	3,761.40	3,020.20%
DCCA	Python	4,675.17	3,778.20%
DCCA	R-Java	28,980.00	23,939.82%
DFA	R	309,276.00	256,454.13%

With twelve time series, the optimized **Zig** version still the fastest, followed by the basic **Zig** implementation. The **Zebende** pure **Python** implementation ended in the third position, performing better than the **DCCA-Python** (4th) and the **DCCA-R-Java** (5th) packages. The lists ended with the **DFA** in the 6th position.

Comparing the basic with the optimized **Zig** algorithms using twelve time series the performance increase is 76.64%. Figure 4 shows the comparison between the basic and optimized **Zig** implementations with twelve time series. The results are equivalent to the ones obtained with four time series, reinforcing the capabilities of the optimization technics.

The basic version is available in **Zebende** version 0.1.91 and the optimized implementation in version 0.2.0 of the package. The codes used for testing are available, as jupyter notebooks, in the link below, where they can be tested in Google colab environment:

https://255ribeiro.github.io/paper_zebendelib/notebooks/test_nb.html

5. Summary and discussion

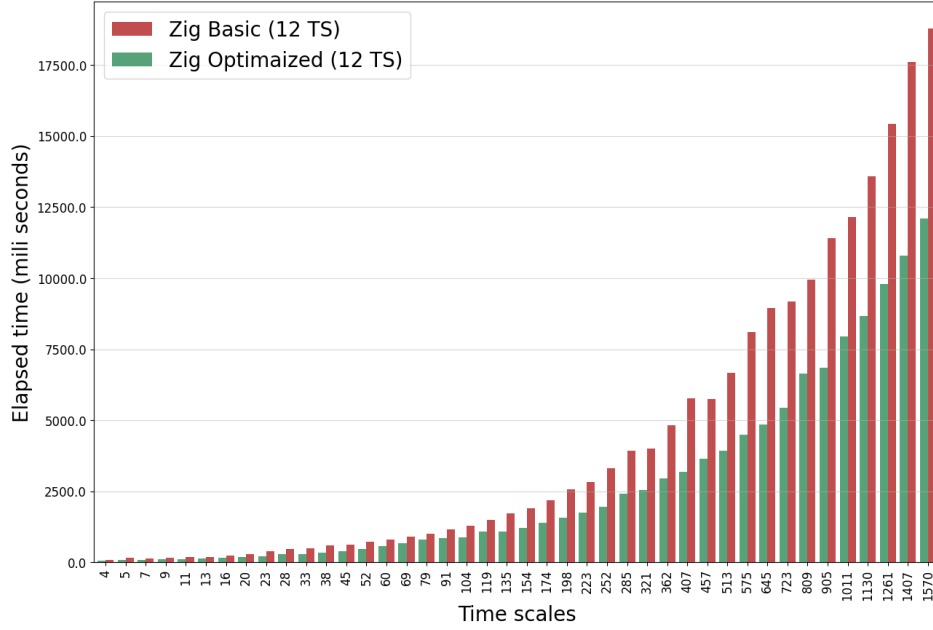


Figure 4: Elapsed Time Comparison for Different Implementations (twelve time series)

The **Zebende** package is the first tool that calculates the DMC_x^2 coefficient for any number of time series. The package also calculates the DFA , $DCCA$ and the ρ_{DCCA} , a fundamental step for the DMC_x^2 calculation. The package was tested with data from the [Ribeiro *et al.* \(2025\)](#) paper, and the results were validated. The package was also compared with other packages that calculate the ρ_{DCCA} , and the results showed that the **Zebende** package is the fastest, making it suitable for large datasets.

The tests with twelve time scales also shows that the **Zebende** implementation, designed to obtain better performance working with greater number of time series, deliver great results even with the pure Python non-optimized version. This package enable the exploration of the DMC_x^2 coefficient in scenarios where it was never tested before. The package is available at the **pypi**.

References

- Alchieri L, Abdalazim N, Alecci L, Gashi S, Gjoreski M, Santini S (2024). “Lateralization Effects in Electrodermal Activity Data Collected Using Wearable Devices.” *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, **8**(1), 1–30.
- Brito AA, Araujo HA, Zebende GF (2019). “Detrended Multiple Cross-Correlation Coefficient applied to solar radiation, air temperature and relative humidity.” *Sci. Rep.*, **9**, 19764. doi: [10.1038/s41598-019-56114-6](https://doi.org/10.1038/s41598-019-56114-6).
- da Silva Filho AM, Zebende GF, de Castro APN, Guedes EF (2021). “Statistical test for multiple detrended cross-correlation coefficient.” *Physica A: Statistical Mechanics and its Applications*, **562**, 125285.

- Ferreira P, Dionísio A, Guedes EF, Zebende GF (2018). “A sliding windows approach to analyse the evolution of bank shares in the European Union.” *Physica A: Statistical Mechanics and its Applications*, **490**, 1355–1367. ISSN 03784371. doi:10.1016/j.physa.2017.08.095.
- Filho FO, Ribeiro F, Cruz JL, De Castro AN, Zebende G (2023). “Statistical Study of the EEG in Motor Tasks (Real and Imaginary).” **622**, 128802. ISSN 03784371. doi:10.1016/j.physa.2023.128802. URL <https://linkinghub.elsevier.com/retrieve/pii/S0378437123003576>.
- Guedes E, Dionísio A, Ferreira PJ, Zebende GF (2017). “DCCA Cross-Correlation in Blue-Chips Companies: A View of the 2008 Financial Crisis in the Eurozone.” **479**, 38–47. ISSN 03784371. doi:10.1016/j.physa.2017.02.065. URL <http://dx.doi.org/10.1016/j.physa.2017.02.065>.
- Guedes EF, Brito AA, Oliveira Filho FM, Fernandez BF, de Castro AP, da Silva Filho AM, Zebende GF (2018a). “Statistical test for $\Delta\rho$ DCCA cross-correlation coefficient.” *Physica A: Statistical Mechanics and its Applications*, **501**, 134–140. ISSN 03784371. doi:10.1016/j.physa.2018.02.148. URL <https://doi.org/10.1016/j.physa.2018.02.148>.
- Guedes EF, Brito AA, Oliveira Filho FM, Fernandez BF, de Castro AP, da Silva Filho AM, Zebende GF (2018b). “Statistical test for $\Delta\rho$ DCCA: Methods and data.” *Data in Brief*, **18**, 795–798. ISSN 23523409. doi:10.1016/j.dib.2018.03.080.
- Guedes EF, da Silva Filho AM, Zebende GF (2021). “Detrended multiple cross-correlation coefficient with sliding windows approach.” *Physica A: Statistical Mechanics and its Applications*, **574**, 125990.
- Guedes EF, de Castro APN, da Silva Filho AM, Zebende GF (2022). “ Δ DMC x 2: A New Approach to Measure Contagion Effect on Financial Crisis.” *Fluctuation and Noise Letters*, **21**(04), 2250026.
- Kacs D, Lee J, Zarins J, Brown N (2024). “Pragma driven shared memory parallelism in Zig by supporting OpenMP loop directives.” In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 930–938. doi:10.1109/SCW63240.2024.00132.
- Mesquita VB, Filho FMO, Danilevicz IM (2024). *DFA: Detrended Fluctuation Analysis*. R package version 1.0.0, URL <https://CRAN.R-project.org/package=DFA>.
- Peng CK, Buldyrev SV, Havlin S, Simons M, Stanley HE, Goldberger AL (1994). “Mosaic Organization of DNA Nucleotides.” **49**(2), 1685–1689.
- Podobnik B, Jiang ZQ, Zhou WX, Stanley HE (2011). “Statistical tests for power-law cross-correlated processes.” *Phys. Rev. E*, **84**(6), 66118. doi:10.1103/PhysRevE.84.066118. URL <https://link.aps.org/doi/10.1103/PhysRevE.84.066118>.
- Podobnik B, Stanley HE (2008). “Detrended cross-correlation analysis: A new method for analyzing two nonstationary time series.” *Physical Review Letters*, **100**(8). ISSN 00319007. doi:10.1103/PhysRevLett.100.084102. 0709.0281.

- Prass TS, Pumi G (2020). *DCCA: Detrended Fluctuation and Detrended Cross-Correlation Analysis*. R package version 0.1.1, URL <https://CRAN.R-project.org/package=DCCA>.
- Ribeiro FF, de Almeida Brito A, Filho FMO, Cruz JAL, Zebende GF (2025). “DCCA multi cross-correlation analysis applied on EEG signals to study motor activity (Real/Imaginary).” *Biomedical Signal Processing and Control*, **103**, 107419. ISSN 1746-8094. doi:<https://doi.org/10.1016/j.bspc.2024.107419>. URL <https://www.sciencedirect.com/science/article/pii/S1746809424014770>.
- Schalk G, McFarland D, Hinterberger T, Birbaumer N, Wolpaw J (2004). “BCI2000: A General-Purpose Brain-Computer Interface (BCI) System.” *IEEE Transactions on Biomedical Engineering*, **51**(6), 1034–1043. doi:[10.1109/TBME.2004.827072](https://doi.org/10.1109/TBME.2004.827072).
- Vassoler RT, Zebende GF (2012). “DCCA cross-correlation coefficient apply in time series of air temperature and air relative humidity.” *Physica A*, **391**(7), 2438–2443. ISSN 0378-4371. doi:[10.1016/j.physa.2011.12.015](https://doi.org/10.1016/j.physa.2011.12.015). URL <http://dx.doi.org/10.1016/j.physa.2011.12.015>.
- Wang GJ, Xie C, Chen S, Yang JJ, Yang MY (2013). “Random Matrix Theory Analysis of Cross-Correlations in the US Stock Market: Evidence from Pearson’s Correlation Coefficient and Detrended Cross-Correlation Coefficient.” *Physica A: Statistical Mechanics and its Applications*, **392**(17), 3715–3730. ISSN 03784371. doi:[10.1016/j.physa.2013.04.027](https://doi.org/10.1016/j.physa.2013.04.027).
- Zebende GF (2011). “DCCA cross-correlation coefficient: Quantifying level of cross-correlation.” *Physica A: Statistical Mechanics and its Applications*, **390**(4), 614–618. ISSN 03784371. doi:[10.1016/j.physa.2010.10.022](https://doi.org/10.1016/j.physa.2010.10.022). URL <http://dx.doi.org/10.1016/j.physa.2010.10.022>.
- Zebende GF, Aguiar LC, Ferreira P, Guedes EF (2022). “Statistical Approach to Study the Relationship Between Stock Market Indexes by Multiple DCCA Cross-Correlation Coefficient.” *Fluctuation and Noise Letters*, **21**(5), 2250045–1314.
- Zebende GF, Silva AM (2018). “Detrended Multiple Cross-Correlation Coefficient.” *Physica A*, **510**, 91–97. ISSN 0378-4371. doi:[10.1016/j.physa.2018.06.119](https://doi.org/10.1016/j.physa.2018.06.119).

Affiliation:

Fernando Ferraz Ribeiro

Faculty of Architecture

Federal University of Bahia

Caetano Moura street, 121,

40210-905, Salvador-BA, Brazil

E-mail: fernando.ribeiro@ufba.br

URL: <https://arquitetura.ufba.br/>

and

Graduate Program in Modeling in Earth and Environmental Sciences

State University of Feira de Santana

Av. Transnordestina

44036-900, Feira de Santana-BA, Brazil

URL: <https://ppgm.uefs.br/>

and

Senai-Cimatec University

Av. Orlando Gomes, 1845,

41650-010, Salvador-BA, Brazil

URL: <https://www.senaicimatec.com.br/>

Gilney Figueira Zebende

Graduate Program in Modeling in Earth and Environmental Sciences

State University of Feira de Santana

Av. Transnordestina

44036-900, Feira de Santana-BA, Brazil

E-mail: gfigzebende@uefs.br

URL: <https://ppgm.uefs.br/>